📝 Kubernetes ConfigMaps – Interview Notes

1️⃣ What is a ConfigMap?

- **Definition:** A ConfigMap in Kubernetes is a **native object used to decouple configuration data from your application code**.
- **Purpose:**
  - Store **non-sensitive configuration data** in key-value pairs.
  - Ensure **configurations persist independently** of pods.
  - Facilitate **reuse of configuration** across multiple pods.

💡 **Best Practice:** Decoupling configuration from code is considered a Kubernetes best practice.

2️⃣ Key Characteristics

- Stores **non-sensitive data only** (use Secrets for sensitive info).
- ConfigMap **data limit:** 1 MB.
- **Namespace scope:** Pod and ConfigMap must reside in the **same namespace** unless using advanced dynamic fetch via the API.
- Can be set as **immutable** (`immutable: true`), preventing updates without deletion & recreation.

3️⃣ Ways to Use ConfigMaps

ConfigMaps can be referenced in **two main ways**:

a) As Environment Variables

- Use when your application expects **single values** via environment variables.
- Syntax:

```
envFrom:
  - configMapRef:
      name: my-config
```

- **Example:** Inject DB host, log level, or API endpoints directly into containers.

---

b) As Files via Volume Mounts

- Use when your app expects **file-based configuration** (e.g., `.properties` for Spring Boot).
- Steps:
  1. Create a **volume** referencing the ConfigMap.
  2. Mount the volume in the container at a specific path.

```
volumes:
  - name: config-volume
    configMap:
      name: my-config

containers:
  - name: app
    image: my-app
    volumeMounts:
      - name: config-volume
        mountPath: /etc/config
```

- **Result:** Each key in the ConfigMap becomes a **file**, and its value becomes the **file content**.

```
/etc/config/
├── ENV          <- contains env value
├── DB_HOST      <- contains db_host value
└── LOG_LEVEL    <- contains log_level value
```

💡 **Use Case:** Mounting initialization scripts or small configuration files into containers.

4️⃣ ConfigMap Example (YAML)

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: my-config
data:
  ENV: "production"
  DB_HOST: "database.example.com"
  LOG_LEVEL: "DEBUG"
```

- Supports **single-line and multi-line values**.
- Can be referenced **via env variables or mounted as files**.

5️⃣ Advanced Notes

- **Dynamic cross-namespace access:** Rare, but possible using the **Kubernetes API** to fetch ConfigMaps from another namespace.
- **Immutable ConfigMaps:**
  - `immutable: true` → prevents updates.
  - To update, **delete & recreate** the ConfigMap.

6️⃣ Summary – When to Use ConfigMaps

Use Case	Approach
Single value configs (DB_HOST, ENV)	Environment variables
File-based configs (application.properties)	Volume mount as file
Small scripts/init scripts	Volume mount as file
Reusable configuration across pods	ConfigMap reference
Immutable, versioned configs	`immutable: true`

✅ **Takeaway:** Always look for opportunities to **decouple configuration from containers**, making apps **more flexible and portable**.

💡 ASCII Diagram: ConfigMap Flow

```
+-------------------+        +--------------------+
|    ConfigMap      |        |     Pod / Container|
|-------------------|  --->  |--------------------|
| ENV=production    |        | ENV=production     |
| DB_HOST=localhost |        | DB_HOST=localhost  |
| LOG_LEVEL=DEBUG   |        | LOG_LEVEL=DEBUG    |
+-------------------+        +--------------------+


or as volume:


+-------------------+        /etc/config/
|    ConfigMap      |  --->  -----------------
| ENV=production    |        | ENV         |
```

```
| DB_HOST=localhost |        | DB_HOST     |
| LOG_LEVEL=DEBUG   |        | LOG_LEVEL   |
+-------------------+        -----------------
```

Here's a detailed, **interview-ready note version** of the update you made to the Color API, structured with **emojis, diagrams, and expanded explanations**:

📝 Color API Update – Supporting Config via Env & File

1️⃣ Objective

Enhance the **Color API** to allow **dynamic configuration** of the default color:

- Accept color via **environment variable** (`DEFAULT_COLOR`)
- Accept color via **file content** (`COLOR_CONFIG_PATH`)
- **Fallback**: Default to `"blue"` if neither is provided

> 💡 This enables **ConfigMap integration** in Kubernetes later, allowing the color to be injected dynamically.

2️⃣ Required Node.js Packages

```
const fs = require('fs');      // File system operations
const path = require('path');  // Path resolution
```

- **fs** → read file contents
- **path** → ensure proper file path resolution across OS

3️⃣ Implementing `getColor()` Function

Step 1: Initialize from Environment Variable

```
let color = process.env.DEFAULT_COLOR;
```

- Checks if user provided a `DEFAULT_COLOR` environment variable
- If not, `color` will be `undefined` for now

Step 2: Check for Config File Path

```
const filePath = process.env.COLOR_CONFIG_PATH;
```

- Optional: If user provides a path, override the color using the file contents

---

Step 3: Read File Contents Safely

```
try {
    const colorFromFile = fs.readFileSync(path.resolve(filePath), 'utf8');
    color = colorFromFile.trim(); // remove line breaks & whitespace
} catch (error) {
    console.error(`Failed to read contents of ${filePath}`, error);
}
```

✅ Features:

- Synchronous read (`readFileSync`) ensures color is available before usage
- `trim()` avoids trailing whitespace or newlines
- Errors are **logged**, but don't crash the app

Step 4: Fallback Default

```
if (!color) {
    color = "blue";
}
return color;
```

- Ensures **robustness**: app always returns a valid color

4️⃣ Updated Function Call

```
const currentColor = getColor();
```

- Replace previous static color assignment
- Now supports **dynamic configuration** via Env or File

5️⃣ Docker & Version Update

- Current version: **1.2.1**
- New version: **1.3.0** (minor version bump due to added feature)

Commands

```
# Build new Docker image
docker build -t lmacademy/color-api:1.3.0 .

# Push to repository
docker push lmacademy/color-api:1.3.0
```

> 💡 **Note:** This change is backward-compatible. If no env/file is provided, the color defaults to blue.

---

6 ASCII Diagram: Flow of `getColor()`

```
+--------------------------+
| Start getColor()         |
+--------------------------+
           |
           v
+--------------------------+
| Check DEFAULT_COLOR env  |
+--------------------------+
           |
           v
+--------------------------+
| Check COLOR_CONFIG_PATH  |
| If file exists → read it |
+--------------------------+
           |
           v
+--------------------------+
| Trim & assign color      |
+--------------------------+
           |
           v
+--------------------------+
| If color undefined       |
```

```
| → fallback to "blue"    |
+-------------------------+
           |
           v
+-------------------------+
| Return color            |
+-------------------------+
```

Here's a **comprehensive, interview-ready note version** of your ConfigMap → Environment Variable workflow, expanded with **diagrams, examples, and explanations**:

📝 Using ConfigMaps as Environment Variables in Kubernetes

1️⃣ Objective

- Pass configuration data from **ConfigMaps** into containers **as environment variables**.
- Demonstrates **decoupling config from application code**.
- Supports **dynamic updates** without rebuilding the container image.

2️⃣ Create ConfigMap YAML

- File: `red-config.yaml`
- API version: `v1`, Kind: `ConfigMap`
- Metadata: `name: red-config`
- Data: `color: red` (key-value pair)

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: red-config
data:
  color: "red"
  multi-line: |
    I AM multi-line
```

💡 Notes:

- Multi-line values can be defined using `|` in YAML.
- Multi-line keys are useful if you want to mount the ConfigMap **as a file**.

3️⃣ Apply ConfigMap

```
kubectl apply -f red-config.yaml
kubectl get configmap
kubectl get configmap red-config -o yaml
```

- `kubectl get configmap` → shows all ConfigMaps in the namespace.
- `-o yaml` → inspect key-value pairs clearly.

4️⃣ Create Pod YAML with Environment Variables

Option 1: Load Entire ConfigMap (Short Version)

```
apiVersion: v1
kind: Pod
metadata:
  name: red-color-api
spec:
  containers:
  - name: color-api
    image: lmacademy/color-api:1.3.0
    ports:
      - containerPort: 80
    envFrom:
      - configMapRef:
          name: red-config
```

✅ Pros: Quick, short, easy for multiple env vars
❌ Cons: Keys in ConfigMap **must exactly match** application environment variable names

---

Option 2: Load Specific Keys (Verbose, Decoupled)

```
apiVersion: v1
kind: Pod
```

```
metadata:
  name: red-color-api
spec:
  containers:
  - name: color-api
    image: lmacademy/color-api:1.3.0
    ports:
      - containerPort: 80
    env:
      - name: DEFAULT_COLOR           # Environment variable in container
        valueFrom:
          configMapKeyRef:
            name: red-config         # ConfigMap name
            key: color               # Key in ConfigMap
```

✅ Pros: Decouples application env variable names from ConfigMap keys
✅ Supports multiple applications with different env variable naming conventions
❌ Slightly more verbose

---

5 Apply Pod

```
kubectl apply -f red-color-api.yaml
kubectl get pods
kubectl describe pod red-color-api
```

- `describe pod` → shows environment variables populated from ConfigMap keys.

---

6 Expose Pod as Service

```
kubectl expose pod red-color-api --type=NodePort --port=80
minikube service red-color-api
```

- Opens browser → verify **default color is red**
- Confirms **ConfigMap environment injection works**

---

7 Update ConfigMap

- Example: Rename key from `color` → `DEFAULT_COLOR`

- Re-apply:

```
kubectl apply -f red-config.yaml
kubectl delete pod red-color-api
kubectl apply -f red-color-api.yaml
```

- Pod automatically picks up updated values on restart

---

8 ASCII Diagram: ConfigMap → Pod Env

```
+---------------------+
|   ConfigMap         |
|---------------------|
| color: red          |
| multi-line: ...     |
+---------------------+
           |
           v
+---------------------+
|      Pod            |
|---------------------|
| Env Variables:      |
| DEFAULT_COLOR=red   |
+---------------------+
           |
           v
+---------------------+
| Container           |
|---------------------|
| App reads DEFAULT_COLOR
| → returns "red"
+---------------------+
```

---

9 Key Takeaways for Interviews

- ✅ **Two ways** to inject ConfigMap into Pod: `envFrom` (short) vs `env` (verbose, decoupled)
- ✅ **Short version** is quick but requires key names to match app env vars

- ✅ **Verbose version** allows multiple apps to use same ConfigMap with different env variable names
- ✅ Supports **dynamic updates**: re-apply ConfigMap and restart pods to propagate changes
- ✅ **Multi-line values** can be used for scripts or config files

---

This sets the stage for **next steps**: mounting ConfigMaps as **files** in containers, which is ideal for configs like `.properties` or scripts.

Here's a **detailed, structured breakdown** of your lecture on **mounting ConfigMaps as volumes** with best practices, examples, and diagrams for clarity:

📝 Mounting ConfigMaps as Volumes in Kubernetes

1️⃣ Objective

- Learn how to **mount ConfigMaps as files inside containers**.
- Understand the behavior when mixing **environment variables** and **file mounts**.
- Explore best practices for **separating concerns** in ConfigMaps.

2️⃣ Create ConfigMap with Mixed Data

File: `green-config.yaml`

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: green-config
data:
  color.txt: "green"
  hello-from-green.js: |
    console.log("Hello from Green ConfigMap");
  color_config_path: "/mount/config/color.txt"
```

- `color.txt` → plain file content
- `hello-from-green.js` → multi-line script

- `color_config_path` → environment variable pointing to file path

💡 **Tip:** Keys in ConfigMaps become filenames when mounted as volumes.

3️⃣ Pod YAML with Mixed ConfigMap Usage

Environment Variable + Volume Mount

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: green-color-api
spec:
  containers:
  - name: color-api
    image: lmacademy/color-api:1.3.0
    ports:
      - containerPort: 80
    env:
      - name: COLOR_CONFIG_PATH
        valueFrom:
          configMapKeyRef:
            name: green-config
            key: color_config_path
    volumeMounts:
      - name: color-config
        mountPath: /mount/config
        readOnly: true
  volumes:
    - name: color-config
      configMap:
        name: green-config
```

How it works:

- **Environment variable:** `COLOR_CONFIG_PATH` → `/mount/config/color.txt`
- **Volume mount:** `/mount/config` contains all keys from ConfigMap as files:

- `color.txt → "green"`
- `hello-from-green.js → "console.log(...)"`
- `color_config_path → "/mount/config/color.txt"`

---

4 Verify Inside Container

```
kubectl exec -it green-color-api -- sh
cd /mount/config
ls
# Output: color.txt  hello-from-green.js  color_config_path
cat color.txt
# Output: green
node hello-from-green.js
# Output: Hello from Green ConfigMap
```

- Files are **read-only**, reflecting the ConfigMap contents.
- Attempting to write to them will fail.

---

5 Expose Pod as Service

```
kubectl expose pod green-color-api --type=NodePort --port=80
minikube service green-color-api
```

- Opens browser → verifies **green color is displayed**.
- Confirms ConfigMap values **mounted correctly as files and env vars**.

---

6 Best Practices: Separate Environment Variables and Files

Option 1: Single Mixed ConfigMap

- Pros: simpler file, fewer ConfigMaps
- Cons: mixing env vars and files can become confusing

Option 2: Split ConfigMaps (Recommended)

```
# green-env-vars.yaml
apiVersion: v1
kind: ConfigMap
metadata:
```

```
  name: green-env-vars
data:
  COLOR_CONFIG_PATH: "/mount/config/color.txt"

# green-files.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: green-files
data:
  color.txt: "green"
  hello-from-green.js: |
    console.log("Hello from Green ConfigMap");
```

- **Pod YAML Changes**:
  - `envFrom:` → load all env vars from `green-env-vars`
  - `volumeMounts:` → mount `green-files` at `/mount/config`

✅ Pros: clear separation, easier maintenance, less risk of confusion

---

7 Key Takeaways

1. **Mounting ConfigMaps as volumes** converts **keys** → **filenames** and **values** → **file contents**.
2. You can pass **specific keys as environment variables** using `valueFrom: configMapKeyRef`.
3. **Volumes are read-only** by default; you cannot modify the mounted files inside the container.
4. **Separate ConfigMaps** for env vars vs. file content improves clarity and maintainability.
5. Works well for:
   - Passing configuration files (`.txt`, `.js`, `.json`)
   - Initialization scripts
   - Environment-specific settings without rebuilding container images

---

8 ASCII Diagram: Mixed ConfigMap Usage

```
+----------------------+
```

```
|      ConfigMap       |
|----------------------|
| color.txt             |
| hello-from-green.js   |
| color_config_path     |
+----------------------+
           |
           v
+----------------------+
|        Pod           |
|----------------------|
| ENV: COLOR_CONFIG_PATH=/mount/config/color.txt
| Volumes:
| /mount/config/
|   color.txt
|   hello-from-green.js
|   color_config_path
+----------------------+
           |
           v
+----------------------+
|      Container       |
|----------------------|
| Reads env var → COLOR_CONFIG_PATH
| Executes scripts → node hello-from-green.js
+----------------------+
```

This pattern is **powerful** for Kubernetes apps that need a mix of:

- Environment variables
- Configuration files
- Initialization scripts

It also scales well when you split ConfigMaps by **concern**.

Here’s a structured summary of your lecture on **Kubernetes Secrets**, highlighting the key points and best practices:

🔐 Kubernetes Secrets Overview

1️⃣ What Are Secrets?

- **Secrets** store **sensitive information** in Kubernetes, such as:
  - Passwords
  - API keys
  - Tokens
  - TLS certificates
- Purpose: **Decouple sensitive data from application code**.

2️⃣ Important Characteristics

- **Stored in base64 encoded format**:
  - ⚠️ **Not encrypted by default**.
  - Base64 is only encoding, not security.
  - Anyone with access to the base64 string can decode it easily:

    ```
    echo <base64-string> | base64 --decode
    ```
- **Best practice**:
  - Enable **encryption at rest** for secrets.
  - Apply **Role-Based Access Control (RBAC)**:
    - Only authorized users/services should list, get, or update secrets.

3️⃣ How Secrets Are Used

- **Similar to ConfigMaps**:
  - Can be injected as **environment variables**.
  - Can be mounted as **files in volumes**.
- Example of usage:

  ```
  env:
    - name: DB_PASSWORD
  ```

```
      valueFrom:
        secretKeyRef:
          name: db-secret
          key: password
```

- Must reside in the **same namespace** as the pod.

---

4 Secret Types

- **Generic secrets (type: `Opaque`)**:
  - Arbitrary key-value pairs.
- **Specialized secret types**:
  - `kubernetes.io/service-account-token` → service account tokens
  - `kubernetes.io/tls` → TLS certificates
  - `kubernetes.io/dockerconfigjson` → Docker registry credentials
- **Cloud provider integration**:
  - Managed Kubernetes often integrates with **native secrets managers** (e.g., AWS Secrets Manager, Azure Key Vault, GCP Secret Manager).

---

5 Sample Secret Definition

```
apiVersion: v1
kind: Secret
metadata:
  name: db-secret
type: Opaque
data:
  username: YWRtaW4=       # base64 encoded 'admin'
  password: c2VjdXJlUGQ=   # base64 encoded 'securePd'
```

- **Notes**:
  - `type: Opaque` → generic secret.
  - Keys (`username`, `password`) are referenced inside pods.
  - Base64 encoding allows storage of arbitrary text safely in YAML.

6️⃣ Best Practices

1. Always **enable encryption at rest** for secrets.
2. Apply **RBAC rules** for access control:
    - Only authorized services/pods should access secrets.
3. Keep **namespace consistency**:
    - Pods and secrets must reside in the **same namespace**.
4. Prefer **managed secret stores** when using cloud providers for additional security.

Secrets in Kubernetes are **powerful** and very similar to ConfigMaps but are **designed for sensitive data**. Understanding encoding vs. encryption and proper RBAC is key to secure usage.

Here's a structured breakdown of your lecture on **working with Kubernetes secrets** using the CLI and pods:

🔐 Working with Secrets in Kubernetes (Practical)

1️⃣ Creating Secrets

While Kubernetes secrets **can be created via YAML manifests**, it's often safer to use the `kubectl` CLI:

- **Reason**: Avoid committing secrets to version control.
- Secrets stored in YAML would be **base64 encoded**, which is **not encryption**.

Example: Create a Generic Secret via CLI

```
kubectl create secret generic db-creds \
  --from-literal=username=db_user \
  --from-literal=password=db_pass
```

- `generic` → arbitrary key-value pairs
- `--from-literal` → directly pass values

Viewing Secrets

- **List secrets**:

```
kubectl get secret
```

- **Describe secret** (shows only metadata and key sizes, not actual values):

```
kubectl describe secret db-creds
```

- **Retrieve values in human-readable form**:

```
kubectl get secret db-creds -o yaml
echo "base64_string_here" | base64 --decode
```

⚠️ Base64 is reversible; anyone with access can decode it.

2️⃣ Using Secrets in Pods (Environment Variables)

Secrets can be injected into pods as environment variables, just like ConfigMaps.

Example: Pod Manifest Using Secret Keys

```
apiVersion: v1
kind: Pod
metadata:
  name: demo-pod
spec:
  containers:
  - name: busybox
    image: busybox:1.36.1
    command: ["sh", "-c", "echo $DB_USER && echo $DB_PASSWORD && sleep 1800"]
    env:
      - name: DB_USER
        valueFrom:
          secretKeyRef:
            name: db-creds
            key: username
      - name: DB_PASSWORD
        valueFrom:
          secretKeyRef:
            name: db-creds
```

```
        key: password
```

- The **keys** in the secret (`username`, `password`) map to **environment variables**.
- The values are **decoded automatically** when injected.

---

2 a Injecting All Secret Keys at Once

Instead of mapping each key individually, you can inject all keys from a secret:

```
envFrom:
  - secretRef:
      name: db-creds
```

- All key-value pairs in the secret become environment variables in the container.
- Key names become **environment variable names** (case-sensitive).

---

3 Key Points

- Secrets are **decoded when injected** into the container:
  - Applications receive the real value, **not base64**.
  - The application must handle secrets carefully (avoid logging them).
- Secrets can be injected **per key** or **all at once** (`envFrom`).
- **Namespace matters**: Pod and Secret must be in the **same namespace**.

✅ Summary

- Use **CLI with literals** to avoid committing secrets in files.
- Use `secretKeyRef` for specific keys or `envFrom` to mount all keys.
- Secrets are **base64 encoded at rest** by default.
- Applications must handle secrets carefully since they are **plain text in memory**.

Here's a clear summary and structured explanation of your lecture on **mounting Kubernetes secrets as volumes in pods**:

🔐 Mounting Secrets as Volumes in Kubernetes

Secrets can be injected into pods **not only as environment variables** but also as **files in a volume**, similar to ConfigMaps. This is useful when your application expects files (like certificates or credentials) instead of environment variables.

1️⃣ Basic Secret Volume Mount

Pod Manifest Example

```
apiVersion: v1
kind: Pod
metadata:
  name: demo-pod
spec:
  containers:
  - name: busybox
    image: busybox:1.36.1
    command: ["sleep", "1800"]
    volumeMounts:
    - name: db-secrets
      mountPath: /etc/db
  volumes:
  - name: db-secrets
    secret:
      secretName: db-creds
```

- **volumeMounts**: defines where the secret will be available inside the container.
- **volumes**: defines the source of the data (here the `db-creds` secret).
- **Result**: each key in the secret becomes a **file** under `/etc/db`:

```
/etc/db/username
/etc/db/password
```

- Contents are **plain text**, not base64-encoded inside the container.

2️⃣ Accessing the Secret in the Pod

```
kubectl exec -it demo-pod -- sh
```

```
cd /etc/db
cat username   # prints db_user
cat password   # prints db_pass
```

⚠️ **Security note**: Anyone who can `exec` into the pod can read secrets. Protect pod access using RBAC.

---

2 Mounting Specific Keys Only

You don't have to mount the entire secret. Use the `items` option to pick specific keys and map them to custom paths.

```
volumes:
  - name: db-secrets
    secret:
      secretName: db-creds
      items:
        - key: password
          path: dev/password
```

- **Result**: only the `password` key is mounted, under `/etc/db/dev/password`.

```
kubectl exec -it demo-pod -- sh
cat /etc/db/dev/password  # prints db_pass
```

- Works the same way for ConfigMaps as well.

---

3 Key Points & Best Practices

- Secrets in volumes are **mounted as files in plain text**.
- Protect pod access using **RBAC**—users with `exec` privileges can read secrets.
- You can selectively mount only specific keys to limit exposure.
- Paths can be customized for better organization inside containers.
- Mounting secrets as volumes is ideal for:
    - TLS certificates
    - Initialization scripts
    - Applications expecting file-based credentials

4️⃣ Cleanup Commands

```
kubectl delete pod demo-pod
```

```
kubectl delete secret db-creds
```

This ensures no leftover pods or secrets are exposing sensitive data.

✅ Summary

Secrets can be injected as:

1. **Environment variables** (decoded automatically)
2. **Files via volume mounts** (plain text in the container)

Volume mounts provide more flexibility for applications that expect files, but you must carefully control **who can access the pod**.